CodeMentor-LLM Project Report

End-to-end data, model training, evaluation, API, database, interface, and deployment

This report consolidates the project documentation, configuration, source code, evaluation artifacts, and deployment notes in the repository. CodeMentor-LLM is a coding assistant built by adapting Meta Llama 3.2 3B Instruct with QLoRA supervised fine-tuning (SFT), followed by Direct Preference Optimization (DPO), and serving the merged model through a Streamlit-to-FastAPI-to-Modal path. Documented results are distinguished from settings directly confirmed by code; see the consistency notes.

1. Project at a glance

Area	Implementation
Purpose	Answer programming questions with concise explanations and working code examples.
Base model	meta-llama/Llama-3.2-3B-Instruct
Training path	CodeAlpaca-20K → cleaning and Llama chat formatting → fixed train/validation/test split → QLoRA SFT → preference pairs → DPO → model merge
Serving path	Browser → Streamlit UI (Hugging Face Spaces) → FastAPI API (Hugging Face Spaces) → Modal A10G GPU → merged model on Hugging Face Hub
Persistence	SQLite + SQLAlchemy stores inference prompt, response, latency, timestamp, and success flag.
Main technologies	PyTorch, Transformers, PEFT, bitsandbytes, TRL, Hugging Face Datasets/Hub, Weights & Biases, FastAPI, Pydantic, SQLAlchemy, SQLite, Streamlit, Modal, Docker.

2. Data collection, preparation, and splits

The primary instruction corpus is sahil2801/CodeAlpaca-20k (documented as 20,022 examples; instruction, input, output; Apache 2.0). The backup source is iamtarun/python_code_instructions_18k_alpaca (18,612 examples), documented as a fallback and not used in the main SFT run. Raw and processed snapshots are versioned on the Hugging Face Hub.

Formatting combines a non-empty input with its instruction and maps each example to a three-message Llama chat template: a fixed coding-assistant system prompt, user instruction, and assistant target. The resulting text field is the model training input.

Cleaning step	Rule and recorded outcome
Null/empty removal	Drop missing or empty text; 0 removed.
Exact deduplication	Deduplicate on text; 0 removed.
Token length	Keep 10–2,048 tokens; recorded corpus range 66–1,194, mean 136.79; 0 removed.
Low-quality filter	Remove examples whose parsed assistant response has fewer than 3 words; 998 removed.
Final clean set	19,024 of 20,022 retained (95.02%); text column; hub snapshot Abdulmoiz123/codementor-llm-cleaned.

The cleaned corpus was split reproducibly with seed 42: 80% train (15,219), 10% validation (1,902), and 10% test (1,903), published as Abdulmoiz123/codementor-llm-splits. SFT code subsequently selects a subset of these splits because of the documented free-T4 resource limit.

3. Model architecture and training

The base is a causal language model. QLoRA loads 4-bit NF4 weights with bfloat16 compute and double quantization, prepares the quantized model for k-bit training, then attaches trainable LoRA adapters. Adapters target q_proj, k_proj, v_proj, and o_proj; rank r=16, alpha=32, dropout=0.05, bias=none, task_type=CAUSAL_LM. The adapter metadata confirms these targets and values. The YAML's stated trainable/total parameter counts are 13,631,488 / 8,043,892,736 (0.17%).

Setting	SFT	DPO
Initialization	Llama-3.2-3B-Instruct + QLoRA	Base model + SFT adapter, trainable
Data	Hub split; script selects 5,000 train examples and 500 validation examples	Abdulmoiz123/codementor-llm-preference; docs record 200 pairs
Epochs	3	1
Per-device batch / accumulation	4 / 4	4 / 4
Effective batch (single device)	16	16
Learning rate / warmup	2e-4 / 0.05 ratio	5e-5 / not specified
Sequence limits	2,048 tokens	512 total; 256 prompt
Optimizer / scheduler	paged_adamw_32bit / cosine	paged_adamw_32bit / cosine

Setting	SFT	DPO
Precision / logging	fp16; every 10 steps; eval and save each epoch	fp16; every 10 steps; save each epoch
Preference control	—	DPO beta = 0.1
Tracking and output	W&B project codementor-llm; push SFT adapter to HF Hub	W&B project codementor-llm; push DPO adapter to HF Hub

The preference construction notebook/code describes chosen answers from the SFT model and rejected answers from the base model, with an intended sample of 200. The DPO script loads the published preference dataset without applying a local row cap. DPO is initialized from the SFT adapter, so it is a second alignment stage rather than training from the original base alone.

4. Evaluation and reported outcomes

The repository stores evaluations on 50 test examples. Metrics include ROUGE-1/2/L and BERTScore precision/recall/F1. These are small-sample, lexical/semantic similarity measures and do not establish broad correctness or production reliability.

Model	ROUGE-1	ROUGE-2	ROUGE-L	BERTScore F1
Base	0.2408	0.0897	0.1807	0.8385
SFT	0.2639	0.1017	0.2004	0.8504
DPO	0.2542	0.1090	0.2077	0.8491

On this recorded run, SFT has the highest ROUGE-1 and BERTScore F1; DPO has the highest ROUGE-2 and ROUGE-L. Earlier SFT-only results also show ROUGE gains over base, while BERTScore F1 is effectively unchanged (0.8366 base vs 0.8363 SFT). Qualitative notes report concise code responses as a strength and repetition, complex-task hallucinations, overlong answers, and truncation as weaknesses. DPO training notes record loss 0.0056, 12 steps, and about 1.5 minutes; SFT notes report 1h 54m on a free T4 and 36.7 MB adapter. These are project log values, not independently reproduced here.

5. Inference, application architecture, and deployment

The merged model is published as Abdulmoiz123/codementor-llm-merged. Inference uses 4-bit NF4, bfloat16 compute, autoregressive generation, and a coding-assistant system prompt. YAML defaults are max_new_tokens=512, temperature=0.7, top_p=0.9, do_sample=true, and repetition_penalty=1.3 appears in the local inference/evaluation path. The current Modal serving method instead sets do_sample=false and use_cache=true, without applying temperature/top_p or repetition_penalty; therefore deployed decoding differs from the general config. The current frontend streams one response and always sends a 512-token cap; it has no automatic continuation or classifier step.

Component	Responsibility / documented deployment
Streamlit frontend	Workspace for coding questions, streamed Markdown output, suggestions, theme/appearance and request-log view. Deploy URL: https://abdulmoiz123-codementor-llm-app.hf.space
FastAPI backend	Validates requests, routes standard and streamed generation, logs results. HF Space: https://abdulmoiz123-codementor-llm-api.hf.space; interactive docs at /docs.
Modal GPU service	Serverless inference with NVIDIA A10G (24 GB per project notes), loading merged HF model in 4-bit; documented endpoint: https://moizishere-droid--codementor-llm-generate-endpoint.modal.run.
Local orchestration	docker-compose maps backend host 8000 to container 7860 and frontend host 8501 to container 7861; frontend points at backend service. HF Spaces use Docker per deployment notes.

The API's default loader calls the Modal endpoint; USE_LOCAL_MODEL=true selects the local model loader. FastAPI lifespan creates the SQLite table and invokes the selected loader at startup. Deployment notes give approximate Modal cold start ~30 seconds and warm inference ~2–5 seconds; treat these as recorded estimates, not an SLA.

6. API endpoints and data storage

Method / path	Inputs and behavior	Response
GET /health	No input; API health metadata.	status, model, version
POST /generate	JSON: prompt (1–2,048 characters), max_new_tokens (1–1,024; default 512). Calls model service and logs inference.	response, latency_ms, success
POST /generate/stream	Same request schema; server-sent events with token events, then [DONE]; failures produce error event.	text/event-stream
GET /logs?limit=5	Returns newest inference logs (default limit 5) and total count. Frontend requests up to 50.	logs array and total

SQLite database codementor.db contains inference_logs: id (integer primary key/index), prompt and response (required strings), latency_ms (required float), timestamp (UTC default), and success (boolean default true). SQLAlchemy creates tables on startup, opens a session per request, commits each logged record, and closes the session. This is request logging, not durable conversation/thread storage. The UI's Request Log is described as an admin view without new authorization behavior.

7. Frontend behavior, operations, and limitations

- The Streamlit application provides a coding workspace, streamed answer rendering, suggestion prompts, light/dark appearance, and a request-log screen with prompt, full response, and status. It requests a fixed 512-token budget; long answers can be cut off.
- CI documentation describes GitHub Actions on pushes and pull requests to main: Ruff lint then pytest. The repository snapshot lists backend and frontend tests, but the documented workflow file was not present in the inspected root listing; current CI execution status is therefore unverified.
- The API enables permissive CORS (all origins/methods/headers). The request log is exposed by API and described as an admin view, with no access-control implementation documented in the inspected route code.
- Local SQLite is suitable for a lightweight demo, but the deployment notes suggest PostgreSQL for production scale. Model hosting depends on Hugging Face Hub, Modal, and credentials configured outside source code.
- The API accepts up to 1,024 generated tokens, while the current UI requests at most 512. Stream interruptions show an error and preserve streamed text; normal finish reason is not returned.

8. Source-of-truth notes and repository map

Where recorded prose conflicts with executable configuration, this report gives both values and identifies the code value as the current implementation. Reconcile these before treating the run metadata as a reproducible experiment:

- SFT validation count: docs/11_sft_training.md reports 1,000; backend/src/train_sft.py selects 500 validation examples. The configured split contains 1,902 validation rows.
- SFT train selection: the training script selects the first 5,000 train rows, matching the documented resource-limited run; it does not train on the complete 15,219-row training split.
- DPO pair count: docs/15_dpo_training.md reports 200 pairs and 12 steps; the DPO script loads the published preference dataset in full and does not limit it to 200 locally. Verify the Hub dataset revision to establish the exact run size.
- Evaluation scope: result JSON files use 50 test samples. Evaluation scripts and recorded artifacts describe slightly different comparison runs, so the metric table above uses the stored three-model DPO result file.
- Deployment notes describe both a FastAPI-to-Modal topology and a local-model option. The default backend loader uses Modal; local loading is conditional. Deployment endpoints are copied from repository notes and were not live-checked for availability.
- The production Modal generator uses greedy decoding (do_sample=false), while local inference config enables sampling (temperature=0.7, top_p=0.9) and repetition penalty 1.3. Do not assume the two paths produce identical outputs.
- The API route contains production-loader code as comments, but the active route imports the non-production model_loader functions; that loader delegates requests to Modal. The live architecture described in deployment notes matches the active Modal delegation path.

Project area	Key files
Planning and technical records	README.md; docs/00_FlowChart.md; docs/01–24 phase notes; notebooks/03–22
Data and model	backend/src/config.yaml; backend/src/data_formatter.py; backend/src/data_cleaner.py; backend/src/train_sft.py; backend/src/train_dpo.py; backend/src/evaluate.py; backend/src/merge_model.py
Serving and UI	backend/api/{main,routes,schemas,database,model_loader}.py; backend/src/modal_app.py; frontend/app.py; frontend/DESIGN.md; hf-combined/
Evidence artifacts	data/results/{baseline_results,sft_evaluation_results,dpo_evaluation_results,adapter_config}.json; Deployment_link.md; docker-compose.yml

Plain-Language Glossary

This appendix explains what the report's common machine-learning and software terms mean, why they appear in CodeMentor-LLM, and what the measurements can and cannot tell you.

A. Training and model terms

Term	Meaning and why it is used here
LLM (Large Language Model)	A neural network trained on large amounts of text to predict and generate the next token. Here, Llama 3.2 3B Instruct is the starting model, adapted for coding help.
Fine-tuning	Continuing training of an already-trained model on a narrower dataset or task. It adapts existing language ability instead of training a model from zero.
SFT (Supervised Fine-Tuning)	Training on examples that pair a prompt with a desired answer. The model learns to imitate those answers. This is the project's first adaptation stage, using coding instruction examples.
Epoch	One full pass over the training examples. Three SFT epochs means the selected SFT examples are revisited three times; one DPO epoch means one pass over preference examples. More epochs are not automatically better: they can overfit.
Batch size and gradient accumulation	Batch size is the number of examples processed in one device step. Gradient accumulation adds several step gradients before updating weights. With batch size 4 and accumulation 4, the effective batch is 16 per device.
Learning rate and warmup	Learning rate controls how large each training update is. Warmup gradually raises it at the beginning to make early training more stable. The SFT config uses 2e-4 with 5% warmup; DPO uses 5e-5.
Optimizer and scheduler	The optimizer applies weight updates; AdamW is an adaptive optimizer with weight decay. Paged AdamW helps manage GPU memory spikes. A cosine scheduler gradually lowers the learning rate following a cosine-shaped curve.
FP16 and BF16	16-bit floating-point number formats that use less memory than 32-bit values. FP16 is configured for training; BF16 is the compute type for 4-bit model loading. They are precision formats, not model sizes.
LoRA (Low-Rank Adaptation)	A parameter-efficient fine-tuning method. It freezes the large pretrained weights and learns small low-rank matrices that modify selected layers. This cuts the number of trainable weights and saves memory; the project targets attention projections q/k/v/o.
Rank r, alpha, dropout, adapter	Rank r controls the size/capacity of the LoRA update (here r=16); alpha scales that update (32); dropout (0.05) randomly drops adapter activations during training to reduce overfitting. The adapter is the small learned set of LoRA weights.
QLoRA	Quantized Low-Rank Adaptation: the base model is loaded in 4-bit form and kept frozen while LoRA adapters are trained. This combines 4-bit memory savings with small trainable updates, making fine-tuning possible on a constrained GPU.
Quantization	Representing model weights with fewer bits. It reduces memory use and can make deployment fit a smaller GPU, with some possible loss or change in model behavior. The project uses 4-bit weights.
NF4 (4-bit NormalFloat)	A 4-bit, non-uniform number format designed for weights that are approximately normally distributed. Its levels are allocated to represent that distribution efficiently. The project selects NF4 to compress the frozen base weights for QLoRA, reducing GPU memory while retaining useful model quality.
Double quantization	Quantizes the small scaling constants used by blockwise 4-bit quantization, saving additional memory. It is enabled in this project's configuration.
Causal language model and tokens	A causal model predicts the next token using the tokens before it. A token is a word fragment or symbol, not necessarily a whole word. max_new_tokens caps how many new tokens can be generated.
Sequence length	The maximum token context processed for a training example. SFT allows 2,048 tokens; DPO allows 512 total tokens and up to 256 prompt tokens. Longer examples may be truncated or excluded depending on pipeline behavior.

B. Preference learning and evaluation

Term	Meaning and why it is used here
DPO (Direct Preference Optimization)	A training method that uses pairs of answers labelled preferred (chosen) and less preferred (rejected). It adjusts the model to favor the chosen answer directly, without a separate reward-model training stage. Here it follows SFT and uses beta=0.1.
Chosen and rejected responses; beta	Chosen is the answer the training data says is better; rejected is the less-preferred answer. Beta controls how strongly DPO balances preference learning against staying near the reference model. The project config sets beta to 0.1.
ROUGE	Recall-Oriented Understudy for Gisting Evaluation. A family of text-overlap metrics originally used mainly to compare machine summaries with reference summaries. It is used here to compare generated coding answers with reference answers; a higher overlap score is not proof that code runs or is correct.
ROUGE-1, ROUGE-2, ROUGE-L	ROUGE-1 measures overlapping single words (unigrams); ROUGE-2 measures overlapping adjacent word pairs (bigrams); ROUGE-L uses the longest common subsequence, so it reflects shared word order. The project reports all three to compare answer similarity at different granularities.
BERTScore, precision, recall, F1	BERTScore compares contextual token representations from a pretrained language model rather than requiring exact word matches. Precision reflects how much generated content matches reference content; recall reflects how much reference content is covered; F1 combines both. It can recognize paraphrases, but cannot certify factual or code correctness.
Test split and sample size	The test split is held back from training for final evaluation. The repository's metric files report 50 test examples, a small sample that gives an initial comparison but may not represent all coding tasks.
Loss and steps	Loss is the training objective's error signal; lower is not by itself proof of better answers. A step is one optimizer update. The recorded DPO loss/12 steps are experiment notes, not a standalone quality guarantee.

C. Inference and application terms

Term	Meaning and why it is used here
Inference and decoding	Inference is using the trained model to answer a new prompt. Decoding is how token predictions are selected. In this project, the Modal service uses greedy decoding (do_sample=false); the general config's sampling settings are not active there.
Temperature, top-p, sampling	When sampling is enabled, temperature adjusts randomness (lower is more predictable), and top-p keeps a probability-ranked set of likely next tokens until their cumulative probability reaches p. The general config lists temperature 0.7 and top_p 0.9, but Modal currently uses do_sample=false, so they do not control deployed output.
Repetition penalty	A decoding adjustment intended to discourage repeating text. The local inference/evaluation code sets 1.3; the active Modal generation code does not apply it.
Merged model	The final model artifact produced by combining trained adapter updates with the base model weights. The repository documents the merged model on Hugging Face Hub for serving.
Latency	Elapsed time for a response, usually reported in milliseconds. It can vary with prompt length, output length, GPU load, network time, and cold starts.
API, REST, endpoint, JSON	An API lets programs communicate. REST is a common web API style using HTTP methods such as GET and POST. An endpoint is one address/action, such as POST /generate. JSON is the structured request/response format used here.
Server-Sent Events (SSE) and streaming	SSE is a one-way HTTP stream from server to browser. Streaming sends generated text in chunks as it becomes available instead of waiting for the complete answer; the project's stream ends with a [DONE] marker.
FastAPI and Pydantic	FastAPI is the Python web framework that hosts the API. Pydantic defines and validates request/response data, such as prompt length and allowed token limits.
SQLite and SQLAlchemy	SQLite is a small database stored in a local file; SQLAlchemy is the Python ORM that maps code objects to database tables. This project uses it to log requests, responses, latency, time, and success status, not to save full conversations.
Streamlit	A Python framework used to build the project's web interface: question entry, streamed answer display, appearance controls, and request log.
Modal, GPU, cold start	Modal hosts the model in an on-demand cloud container. A GPU accelerates model calculations. A cold start is the delay when a new container starts and loads the model; warm requests reuse an already-running container.
Hugging Face Hub and Weights & Biases	Hugging Face Hub stores model and dataset artifacts. Weights & Biases (W&B) records experiment configuration and training metrics.
Docker, CI/CD, CORS	Docker packages an app and its dependencies into a container. CI/CD means Continuous Integration/Continuous Delivery: automated checks/builds on code changes. CORS (Cross-Origin Resource Sharing) is a browser rule that controls which website origins may call an API; the current API config allows all origins.